

UT01 – Tecnologías de programación sobre clientes web

ACTIVIDAD EVALUABLE (RA1) – PLANTILLA DE ENTREGA

**Nombre y Apellidos del
alumno/a:**

Martín López Arecha

Fecha de entrega:

18/09/2026

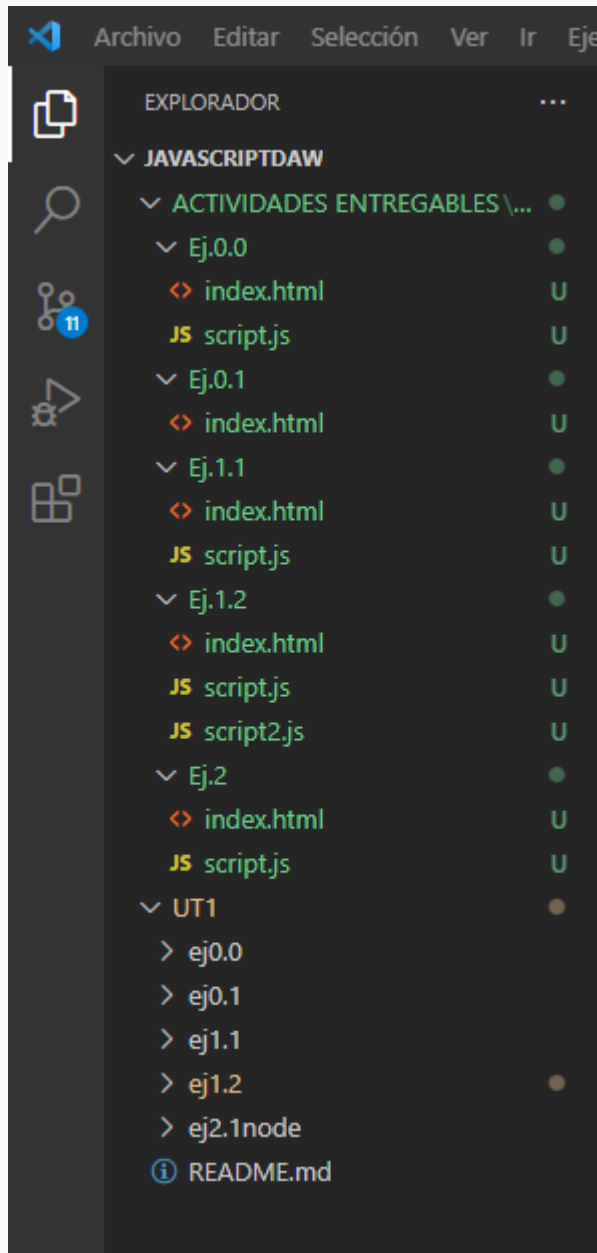
URL del repositorio GitHub:

<https://github.com/martinlopezcla/javascriptDAW.git>

1. Repositorio y Estructura de la Actividad (0,50 pts)

El repositorio de GitHub ha sido configurado bajo la estructura obligatoria **ACTIVIDADES ENTREGABLES / ACT-1 /** y compartido con la profesora (aralvarezr@educa.jcyl.es). A continuación, se presentan las evidencias de código y funcionamiento de cada ejercicio.

 **Evidencia: Captura del explorador de archivos donde se vean todos los archivos generados.**



Ejercicio 1.1 – Carga Diferida con 'defer'

- Estructura: Archivo **index.html** con `<script defer src="..."></script>` ubicado en la cabecera `<head>`. El script genera los números del 1 al 100 en el cuerpo del documento mediante `document.body.innerHTML += "<p>"+i+"</p>"`.

Explica aquí qué ocurre si eliminas el atributo defer de la etiqueta `<script>`

Se ejecutaría el script en cuanto el navegador detecta el head antes de exista document body y lo marca como null y no pone los números.

Ejercicio 1.2 – Carga Asíncrona con 'async'

- Estructura: Archivo **index.html** con dos scripts asíncronos enlazados en el `<head>` mediante `<script async src="..."></script>`. Uno muestra los números del 1 al 100 y el otro del 900 al 1000.

Explica aquí qué se pretende demostrar y si se ha conseguido con este ejemplo

Que los scripts se ejecutan los dos a la vez sin respetar el orden como pasa con defer.

Se ha conseguido, pero no se ve de forma visual porque se ejecuta un script y luego el otro, los números no se intercalan.

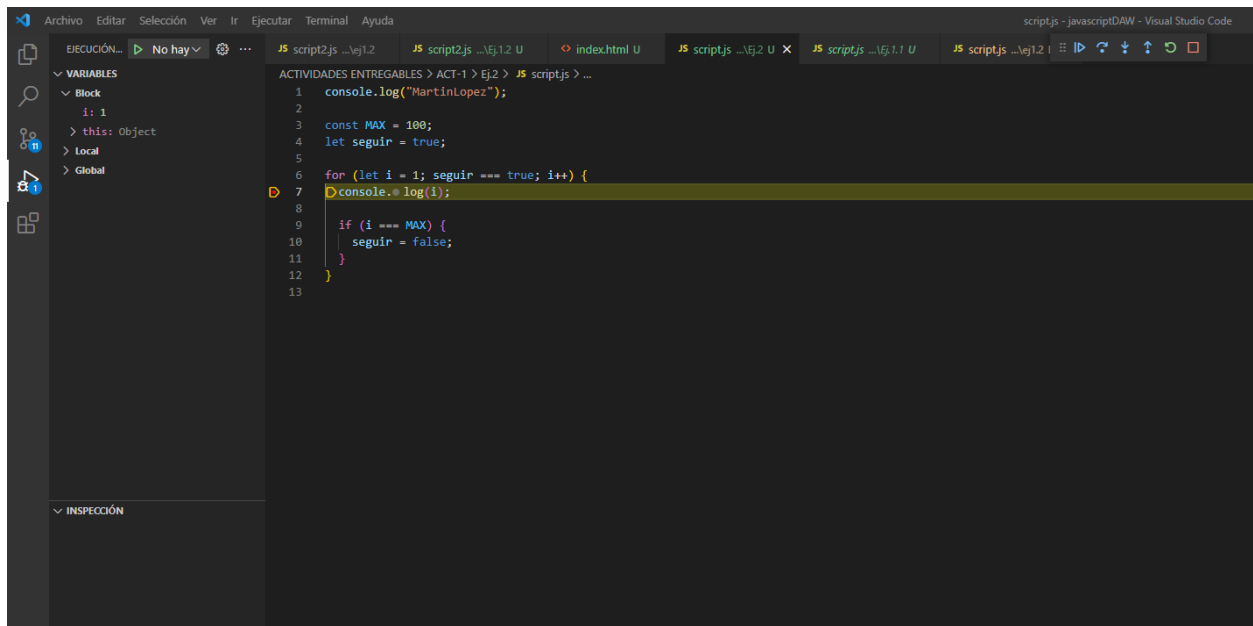
2. Depuración en Visual Studio Code (0,75 pts)

La sesión de depuración se realiza sobre el archivo JavaScript ubicado en la carpeta **Ej.2**. A continuación se documentan los elementos clave de la interfaz de depuración de VS Code.

2.1. Punto de Ruptura (Breakpoint)

Captura de Pantalla: Punto de ruptura (Breakpoint) activo en el código del Ej.2

Al colocar un breakpoint en la línea del bucle la ejecución se detiene justo antes de procesarlo, esto te permite ver el valor de la variable o el estado del programa en un punto concreto antes de continuar. En este caso el valor que tiene i.



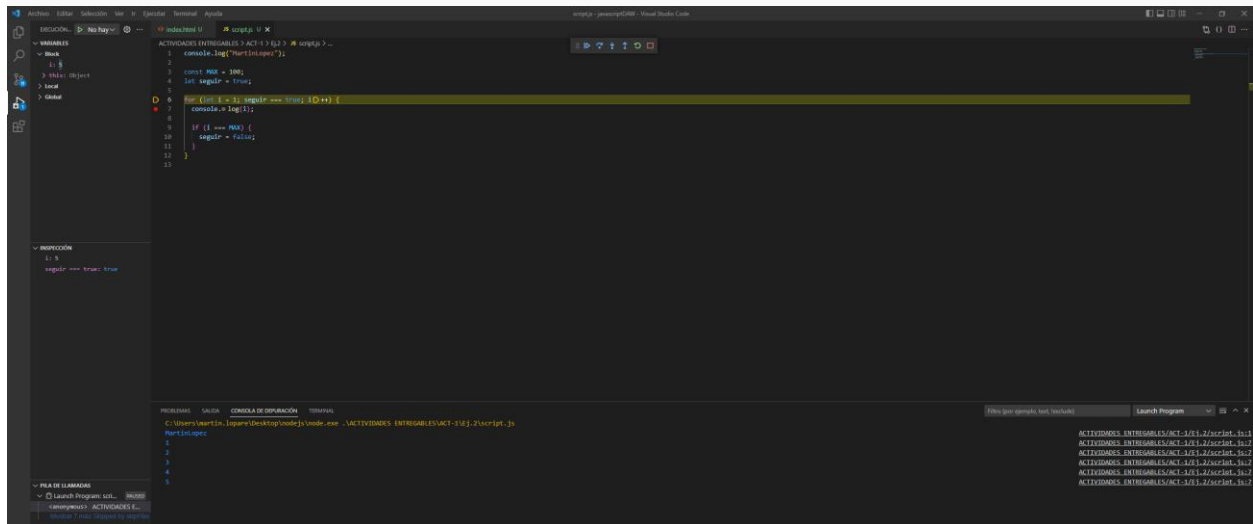
2.2. Botones de Control de Depuración de VS Code

Identificación y explicación técnica del efecto de los 6 botones del panel flotante de depuración:

Botón / Acción	Icono	Atajo de Teclado	Efecto / Descripción Técnica
Continuar (Continue)		F5	Sigue la ejecución normal del programa hasta el siguiente breakpoint o hasta que termine.
Paso a paso por encima (Step Over)		F10	Ejecuta la línea actual completa sin entrar en las funciones que llama, si la línea llama a una función, la ejecuta de una vez y se pasa a la siguiente.
Paso a paso por dentro (Step Into)		F11	Si la línea actual llama a una función, entra dentro de ella y se detiene en su primera línea, permitiendo depurarla paso a paso.
Paso a paso para salir (Step Out)		Shift+F11	Si estas en una función termina de ejecutarla entera y vuelve al punto donde se llamó y se para justo después de esa llamada.
Reiniciar (Restart)		Ctrl+Shift+F5	Para la sesión de depuración actual y la vuelve a lanzar desde el principio, sin salir del modo debug.
Detener (Stop)		Shift+F5	Termina la sesión de depuración y detiene la ejecución del programa.

2.3. Inspección de Código: Zonas Watch y Variables

 **Captura de Pantalla: Panel 'Watch' con expresiones en seguimiento y 'Variables' en un estado concreto**



3. Ejecución con Node.js (0,25 pts)

Ejecución del script de la carpeta Ej2 en el entorno de servidor local Node.js mediante la terminal integrada de Visual Studio Code.

Captura de Pantalla: Terminal integrada con comando 'node -v', la ruta completa y la ejecución del script mostrando la salida personalizada

Se ejecuta el script directamente con Node.js desde la terminal de VS Code, se comprueba la versión instalada con node -v, luego con cd entramos al directorio del script y lo ejecutamos con node script.js.



```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL
PS C:\Users\martin.Iopare\Desktop\javascriptDAM\javascriptDAM> node -v
v24.21.0
PS C:\Users\martin.Iopare\Desktop\javascriptDAM\javascriptDAM> cd "C:\Users\martin.Iopare\Desktop\javascriptDAM\javascriptDAM\ACTIVIDADES ENTREGABLES\ACT-1\Ej.2"
PS C:\Users\martin.Iopare\Desktop\javascriptDAM\javascriptDAM\ACTIVIDADES ENTREGABLES\ACT-1\Ej.2> node script.js
MartinLopez
1
2
3
4
5
6
7
8
9
10
11

```

4. Lista de Cotejo / Autoevaluación

Calificación máxima: 1,50 pts · Escala: Sí = puntuación total · Parcial = 50 % · No = 0

Criterio de Evaluación	Puntuación Máx.	Sí/Parcial /No
1. Repositorio y estructura	0,50	Si
· Repositorio compartido con aralvarezr@educa.jcyl.es y accesible (sin 404).	0,05	Si
· Carpeta ACTIVIDADES ENTREGABLES / ACT-1 con el documento y ejercicios.	0,05	Si
· Ej.0.0: .html con <script src="..."> y script.js que muestra 'Hola mundo'.	0,10	Si
· Ej.0.1: .html con script en línea (sin src) y resultado coincidente con Ej.0.0.	0,10	Si
· Ej.1.1: Script externo con defer en <head> escribe 1-100 en document.body.	0,10	Si

· Ej.1.2: Dos scripts con async en <head>, uno 1-100 y otro 900-1000.	0,10	Si
2. Depuración en Visual Studio Code	0,75	Si
· Puntos de ruptura (breakpoints) con ejecución pausada y línea resaltada.	0,20	Si
· 6 Botones de avance identificados y explicados con sus efectos.	0,20	Si
· Zona Watch con expresión añadida y valor durante la ejecución.	0,15	Si
· Zona Variables comentando cambios de valor en ejecución.	0,15	Si
· Calidad de evidencia: capturas legibles.	0,05	Si
3. Ejecución con Node.js	0,25	Si
· Ejecución del comando en la ruta correcta.	0,10	Si
· Salida por consola correcta, incluyendo el nombre del alumno	0,10	Si
· Entorno visible: terminal integrada de VS Code y versión de Node (node -v).	0,05	Si
CALIFICACIÓN OBTENIDA		

Observaciones:

[Si quieres añadir alguna observación sobre esta actividad, puedes hacerlo aquí]